Instituto Politécnico de Viana do Castelo

Licenciatura em ENGENHARIA INFORMÁTICA / Degree INFORMATICS ENGINEERIN

INTEGRAÇÃO DE SISTEMAS

TP3

Ivo Ramoa

Orientador(es):

- Professor Doutor Jorge Ribeiro, Professor Doutor Jackson Junior

Índice

1. Introdução e Objetivos
2. Ferramentas Utilizadas
3. Estrutura do Projeto
4. Pipeline
   - 4.1 Pipeline - Exemplo de CSV
   - 4.2 Pipeline - Crawler
   - 4.3 Pipeline - Bucket
   - 4.4 Pipeline - Processor
   - 4.5 Pipeline - XML Service
   - 4.6 Pipeline - XPath/XMLTABLE
   - 4.7 Pipeline - BI Service (GraphQL)
   - 4.8 Pipeline - Viewer Service
   - 4.9 Pipeline - Interfaces de Interoperabilidade
   - 4.10 Pipeline - Docker
   - 4.11 Pipeline - Azure
   - 4.12 Pipeline - GitHub
5. Conclusão
6. Bibliografia e Referências Web

Introdução e Objetivos

Este Trabalho Prático 3 tem como objetivo aplicar conceitos de Integração de Sistemas através da construção de uma **solução distribuída, baseada em microserviços, capaz de representar, transformar e disponibilizar dados usando diferentes formatos e protocolos**. A solução implementa um **pipeline completo desde a geração de dados em CSV, o armazenamento em buckets do Supabase** (via S3 API), **o processamento e enriquecimento** com uma **API externa,** até à **conversão para XML e persistência numa base de dados relacional** (PostgreSQL) com suporte para **XML**. Por fim, os **dados persistidos** são disponibilizados através de **múltiplas interfaces de consumo** (REST, GraphQL, XML-RPC e gRPC), permitindo **interoperabilidade entre sistemas** com necessidades distintas.

Introdução e Objetivos

Objetivos:

- **Implementar uma solução com vários serviços independentes**, orquestrados por Docker Compose, garantindo modularidade e reutilização.
- **Utilizar pelo menos três linguagens de programação distintas** no desenvolvimento dos serviços.
- **Armazenar ficheiros CSV num bucket Supabase através da S3 API** e suportar um fluxo periódico de ingestão.
- **Processar os dados de forma assíncrona**, mapeando atributos para um modelo desacoplado e enriquecendo a entidade com dados obtidos numa API externa.

Introdução e Objetivos

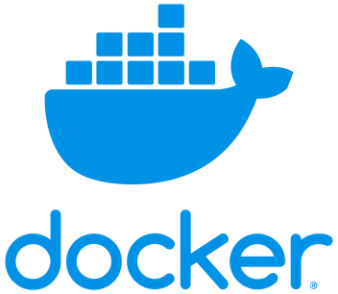
- **Gerar documentos XML com estrutura hierárquica** e persistir os mesmos numa base de dados relacional com suporte para XML.

- **Implementar consultas ao XML persistido através de XPath/XMLTABLE** e expor resultados por endpoints dedicados.

- **Disponibilizar pelo menos três consultas no BI Service** e suportar diferentes protocolos de comunicação, assegurando a interoperabilidade do sistema.

- **Garantir versionamento do código (GitHub) e das imagens Docker (Docker Hub),** com configuração parametrizada via variáveis de ambiente.

Ferramentas Utilizadas

















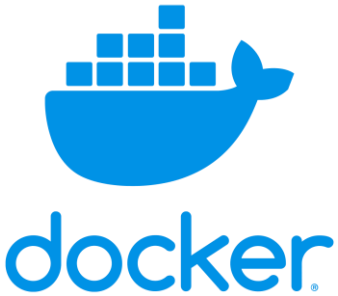






supabase

Ferramentas Utilizadas



Plataforma de contentores que empacota aplicações e dependências. Foi usada para containerizar cada serviço e garantir execução consistente em qualquer máquina.



Plataforma de controlo de versões e colaboração. Foi usada para alojar o repositório e manter o histórico/entrega do código.



Plataforma que fornece, entre outros, Storage compatível com S3. Foi usada como bucket intermédio para troca de ficheiros entre crawler e processor.

Ferramentas Utilizadas



Linguagem versátil para automação, APIs e integração. Foi usada nos serviços de crawler, processor, BI, XML-RPC e gRPC.



Ambiente JavaScript para executar servidores e APIs. Foi usado no xml-service para receber o CSV, gerar XML e comunicar com o PostgreSQL.



Linguagem compilada, eficiente e simples para serviços web. Foi usada para criar o viewer-service, provando a 3.ª linguagem e consumindo o BI Service via GraphQL.



Plataforma de cloud para alojar serviços. Foi usada como ambiente de cloud para armazenar o xml-service.

Ferramentas Utilizadas

gRPC

Protocolo RPC moderno baseado em Protobuf, com contratos fortes. Foi usado para expor métodos remotos de consulta aos dados.

GraphQL

Linguagem de consulta para APIs.Foi usada no BI Service para expor consultas flexíveis sobre os dados do XML Service.

PostgreSQL

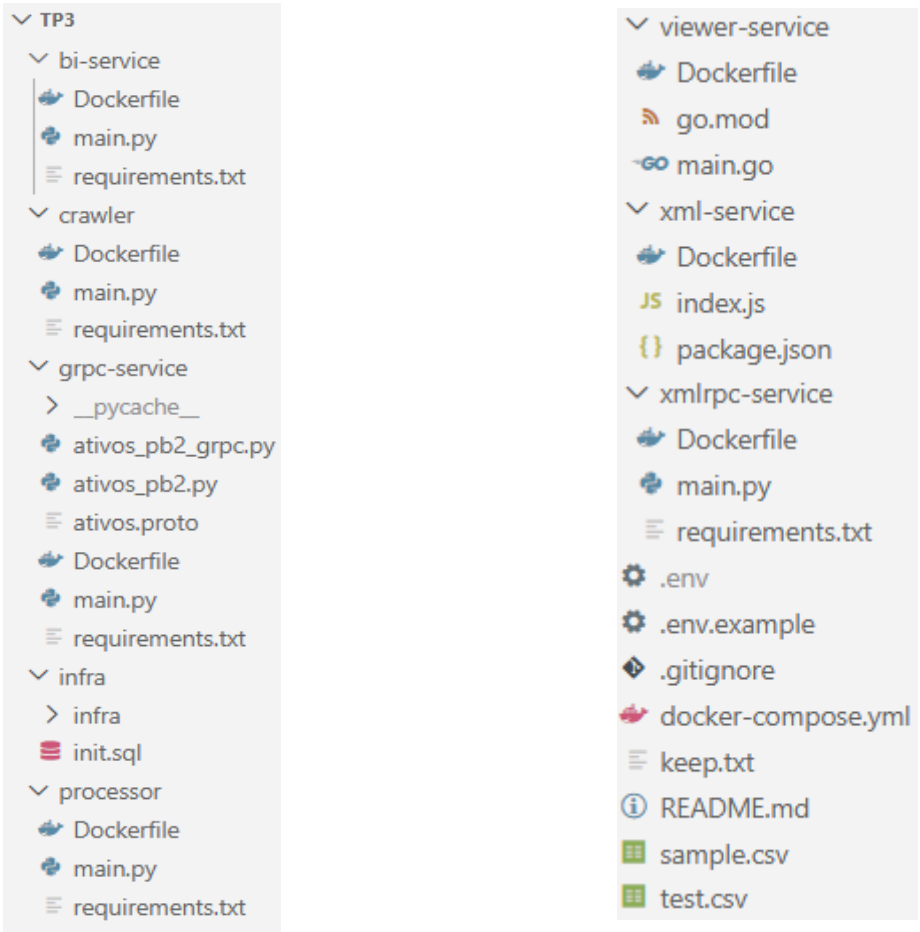
SGBD relacional com suporte para tipo XML e queries sobre XML. Foi usado para persistir os documentos XML e respetiva metadata.

<xpath>

Linguagem para navegar e extrair dados de documentos XML. Foi usada nas queries para consultar o XML e gerar resultados para os endpoints.

Estrutura do Projeto

No fim do trabalho, a estrutura do projeto ficou da seguinte forma:



Pipeline – Exemplo de CSV

```
1   internal_id,symbol,category,price,volume
2   CSV_1001,AAPL,Tecnologia,189.2,1200000
3   CSV_1002,TSLA,Automovel,210.5,800000
```

Na imagem acima pode-se ver um **exemplo de CSV gerado** pelo **crawler**. É composto pelas **colunas internal_id, symbol, category, price, volume,** sendo que este é o input base do pipeline; o **processamento e enriquecimento ocorre no passo seguinte**. É de ressalvar que o trabalho começou com uma amostra pequena (comparada aos ficheiros finais), daí o test.csv e o sample.csv no projeto.

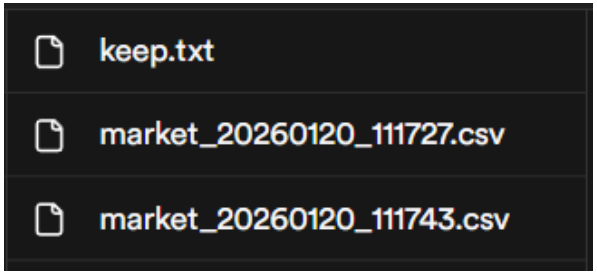
Pipeline – Exemplo de CSV

	A	B	C	D
1	internal_id,symbol,category,price,volume			
2	CSV_3001,MSFT,Tecnologia,113.9,1592944			
3	CSV_3002,GOOGL,Tecnologia,100.79,549125			
4	CSV_3003,AAPL,Tecnologia,312.5,598141			
5	CSV_3004,AMZN,Tecnologia,317.76,773034			
6	CSV_3005,MSFT,Tecnologia,190.8,105981			
7	CSV_3006,AAPL,Tecnologia,272.33,1067082			
8	CSV_3007,AAPL,Tecnologia,143.03,126111			
9	CSV_3008,AAPL,Tecnologia,304.52,1328728			
10	CSV_3009,MSFT,Tecnologia,340.04,436090			
11	CSV_3010,AAPL,Tecnologia,156.23,1457735			
12	CSV_3011,MSFT,Tecnologia,137.65,1620600			
13	CSV_3012,GOOGL,Tecnologia,281.03,830990			
14	CSV_3013,AMZN,Tecnologia,358.45,881239			
15	CSV_3014,MSFT,Tecnologia,104.73,1508077			
16	CSV_3015,TSLA,Automovel,391.56,1375176			
17	CSV_3016,MSFT,Tecnologia,188.58,1039924			
18	CSV_3017,AMZN,Tecnologia,384.67,414779			
19	CSV_3018,AMZN,Tecnologia,367.41,789860			
20	CSV_3019,MSFT,Tecnologia,226.95,818218			
21	CSV_3020,MSFT,Tecnologia,278.62,840975			
22	CSV_3021,MSFT,Tecnologia,171.25,1613069			
23	CSV_3022,GOOGL,Tecnologia,388.64,724820			
24	CSV_3023,AAPL,Tecnologia,315.92,339526			
25	CSV_3024,AAPL,Tecnologia,386.5,623606			

- **internal_id:** identificador interno do registo
- **symbol:** ticker do ativo
- **category:** tipo/setor
- **price:** preço atual
- **volume:** volume negociado

Pipeline - Crawler

O **crawler** é o serviço de origem de dados. A sua função é **gerar periodicamente** um ficheiro CSV com dados de mercado simulados e **enviá-lo para** um **bucket Supabase** via **S3 API**, na **pasta incoming/.** Isto inicia todo o pipeline. É de ressaltar que só são processados ficheiros .csv e o .txt foi colocado ali para servir de exemplo que não é processado.



A cada N segundos (ex.: 15), o **crawler cria o conteúdo CSV em memória, gera um nome único com timestamp**: incoming/market_YYYYMMDD_HHMMSS.csv e **faz put_object para o bucket do Supabase**. **Repete continuamente,** em **loop,** simulando um feed contínuo de dados.

Pipeline - Crawler

Passando para a parte técnica, **primeiramente é definido o cliente S3**, em que são lidas as variáveis do .env, e **cria o clinente**.

```
s3 = boto3.client(
    "s3",
    region_name=S3_REGION,
    aws_access_key_id=S3_ACCESS,
    aws_secret_access_key=S3_SECRET,
    endpoint_url=S3_ENDPOINT,
    config=Config(signature_version="s3v4", s3={"addressing_style": "path"}),
)
```

Pipeline - Crawler

De seguida, **segue para a parte de gerar o CSV**, em que **define as linhas com os campos enunciados, escreve o header e as linhas**, e **devolve uma string CSV.**

```
def generate_csv() -> str:
    rows = [
        {"internal_id":"CSV_3001","symbol":"AAPL","category":"Tecnologia","price":"189.2","volume":"1200000"},
        {"internal_id":"CSV_3002","symbol":"TSLA","category":"Automovel","price":"210.5","volume":"800000"},
        {"internal_id":"CSV_3003","symbol":"MSFT","category":"Tecnologia","price":"310.2","volume":"900000"},
    ]
    out = io.StringIO()
    w = csv.DictWriter(out, fieldnames=["internal_id","symbol","category","price","volume"])
    w.writeheader()
    w.writerows(rows)
    return out.getvalue()
```

Pipeline - Crawler

Por fim, na parte de fazer upload, **cria uma chave com um timestamp** e **faz a função *s3.put_object*** para **enviar o CSV para o bucket.**

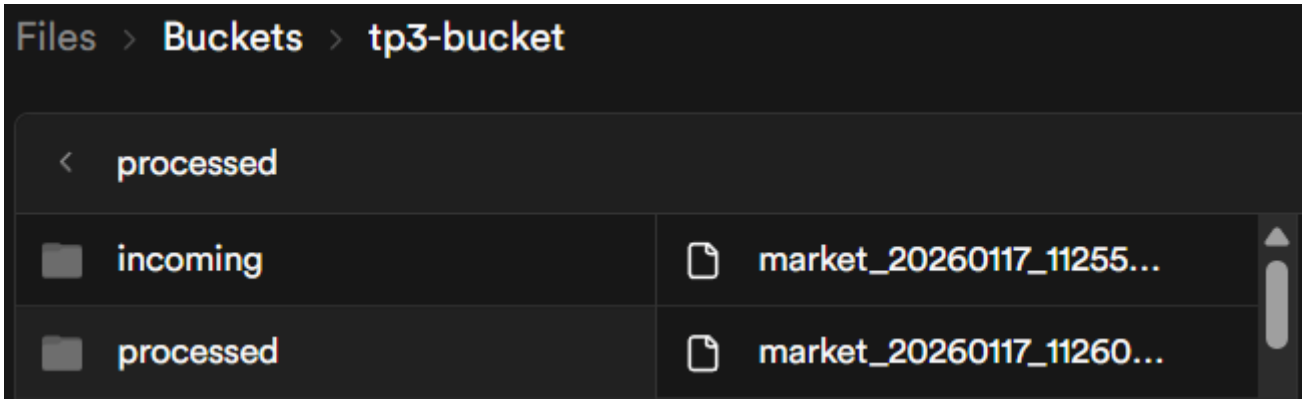
```
def upload_incoming(content: str):
    ts = datetime.datetime.utcnow().strftime("%Y%m%d_%H%M%S")
    key = f"incoming/market_{ts}.csv"
    s3.put_object(Bucket=BUCKET, Key=key, Body=content.encode("utf-8"))
    print("Uploaded:", key)
```

Como se pode ver pela evidência seguinte, o c**rawler gera periodicamente um CSV** e **envia-o para o Supabase,** iniciando o pipeline.

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> docker compose exec crawler python main.py
Uploaded: incoming/market_20260120_144844.csv
```

Pipeline - Bucket

Avançando para a etapa seguinte na Pipeline, vem o **Bucket**. O **bucket funciona como camada intermédia de integração.** O **crawler escreve em incoming/** e o **processor lê, transforma e escreve em processed/.**



Pipeline - Processor

O **processor** é o serviço ETL do sistema. Consome ficheiros CSV colocados no bucket, transforma-os para um formato "mapeado", **enriquece** cada registo com dados de uma **API externa** (Stooq) e entrega o resultado ao xml-service para a geração/persistência do XML. Também gere o **webhook** de confirmação para garantir que o ficheiro de entrada só é removido quando o XML já foi gravado com sucesso.

Pipeline - Processor

O processor faz polling ao bucket e lista os ficheiros em incoming/ a cada intervalo definido por PROCESSOR_POLL_SECONDS. Quando **encontra um CSV**, **faz o download a partir do S3**, **lê o conteúdo e percorre cada linha, aplicando um mapeamento para garantir campos consistentes (internal_id, symbol, category, price e volume).** De seguida, **enriquece cada registo calculando avg30d e max6m através da API externa Stooq,** com cache e fallback para garantir robustez. No fim, **gera um novo CSV já enriquecido e faz upload para processed/.**

1	internal_id,symbol,category,price,volume,avg30d,max6m
2	CSV_3001,AAPL,Tecnologia,189.2,1200000,185.42,198.66
3	CSV_3002,TSLA,Automovel,210.5,800000,206.29,221.03
4	CSV_3003,MSFT,Tecnologia,310.2,900000,304.00,325.71

Pipeline - Processor

	A	B	C	D	E	F
1	internal_id,symbol,category,price,volume,avg30d,max6m					
2	CSV_3001,TSLA,Automovel,159.85,1716897,455.42,489.88					
3	CSV_3002,AMZN,Tecnologia,293.8,530852,232.78,254.00					
4	CSV_3003,TSLA,Automovel,292.87,394781,455.42,489.88					
5	CSV_3004,GOOGL,Tecnologia,351.22,243910,317.31,335.97					
6	CSV_3005,AAPL,Tecnologia,196.41,1166816,268.90,286.19					
7	CSV_3006,TSLA,Automovel,279.28,1166691,455.42,489.88					
8	CSV_3007,AAPL,Tecnologia,330.65,1103268,268.90,286.19					
9	CSV_3008,AMZN,Tecnologia,113.76,1631576,232.78,254.00					
10	CSV_3009,MSFT,Tecnologia,379.88,1815424,478.38,542.07					
11	CSV_3010,GOOGL,Tecnologia,123.56,891129,317.31,335.97					
12	CSV_3011,GOOGL,Tecnologia,341.19,1186070,317.31,335.97					
13	CSV_3012,MSFT,Tecnologia,123.67,523702,478.38,542.07					
14	CSV_3013,AMZN,Tecnologia,376.48,1224606,232.78,254.00					
15	CSV_3014,MSFT,Tecnologia,322.88,1524085,478.38,542.07					
16	CSV_3015,MSFT,Tecnologia,364.42,1957628,478.38,542.07					
17	CSV_3016,MSFT,Tecnologia,323.79,897116,478.38,542.07					
18	CSV_3017,AAPL,Tecnologia,377.05,451761,268.90,286.19					
19	CSV_3018,AAPL,Tecnologia,377.0,182230,268.90,286.19					
20	CSV_3019,GOOGL,Tecnologia,372.15,540198,317.31,335.97					
21	CSV_3020,MSFT,Tecnologia,225.77,1190509,478.38,542.07					
22	CSV_3021,AMZN,Tecnologia,301.11,1611556,232.78,254.00					
23	CSV_3022,MSFT,Tecnologia,264.1,1765222,478.38,542.07					
24	CSV_3023,MSFT,Tecnologia,347.23,1228067,478.38,542.07					
25	CSV_3024,GOOGL,Tecnologia,120.56,1225707,317.31,335.97					
26	CSV_3025,MSFT,Tecnologia,271.6,375966,478.38,542.07					
27	CSV_3026,AMZN,Tecnologia,269.51,278802,232.78,254.00					
28	CSV_3027,AAPL,Tecnologia,201.2,1941406,268.90,286.19					
29	CSV_3028,AAPL,Tecnologia,131.36,678737,268.90,286.19					
30	CSV_3029,AAPL,Tecnologia,194.6,1672568,268.90,286.19					

A ingestão para XML é feita através de um POST /ingest no xml-service, enviando também mapper_version, request_id e webhook_url. **Quando o processor recebe o webhook com status=OK, apaga o ficheiro original em incoming/, evitando reprocessamento do mesmo ficheiro.**

Pipeline - Processor

Na parte do código, funciona da seguinte forma: as duas **funções seguintes procuram continuamente por ficheiros para processar.**

```python
def list_incoming():
    resp = s3.list_objects_v2(Bucket=BUCKET, Prefix="incoming/")
    return [
        o["Key"]
        for o in resp.get("Contents", [])
        if o["Key"].endswith(".csv") and not o["Key"].endswith("/.keep")
    ]
```

```python
def poll_loop():
    while True:
        try:
            keys = list_incoming()
            if keys:
                process_one(keys[0])
        except Exception as e:
            print("poll error:", e)
        time.sleep(POLL_SECONDS)
```

Pipeline - Processor

Na função ***process_one***, **lê o CSV, cria um novo CSV na variável fieldnames, escreve para o processed/ e imprime a mensagem de sucesso.**

```python
def process_one(incoming_key: str):
    print("Processing:", incoming_key)
    text = download_text(incoming_key)

    reader = csv.DictReader(io.StringIO(text))
    out = io.StringIO()
    fieldnames = ["internal_id","symbol","category","price","volume","avg30d","max6m"]
    w = csv.DictWriter(out, fieldnames=fieldnames)
    w.writeheader()

    for row in reader:
        mapped = {
            "internal_id": row.get("internal_id",""),
            "symbol": row.get("symbol",""),
            "category": row.get("category",""),
            "price": row.get("price","0"),
            "volume": row.get("volume","0"),
        }
        w.writerow(enrich_row(mapped))

    processed_key = incoming_key.replace("incoming/", "processed/")
    upload_text(processed_key, out.getvalue())
    print("Uploaded processed:", processed_key)

    request_id = str(uuid.uuid4())
    pending[request_id] = incoming_key
```

Pipeline - Processor

Nas funções abaixo, junto com as funções ***_external_enrich()*** e ***enrich_row()*,** **chamam a Stooq, calculam a média e o máximo, usam cache e se falhar realizam fallback.**

```python
def _fetch_stooq_history_close(symbol: str):
    s = _stooq_symbol(symbol)
    if not s:
        return []

    url = f"https://stooq.com/q/d/l/?s={s}&i=d"
    r = requests.get(url, timeout=20)
    r.raise_for_status()

    # CSV com colunas: Date,Open,High,Low,Close,Volume
    rows = []
    reader = csv.DictReader(io.StringIO(r.text))
    for row in reader:
        try:
            c = float(row.get("Close") or "")
            # ignora valores inválidos
            if c > 0:
                rows.append(c)
        except Exception:
            continue

    return rows
```

```python
def _compute_avg30d_max6m_from_closes(closes: list[float]):
    # 30 dias úteis ~ 30 registos
    last30 = closes[-30:] if len(closes) >= 30 else closes[:]
    # 6 meses úteis ~ 126; usamos 180 para margem
    last6m = closes[-180:] if len(closes) >= 180 else closes[:]

    if not last30 or not last6m:
        return None, None

    avg30 = sum(last30) / len(last30)
    max6m = max(last6m)
    return avg30, max6m
```

Pipeline - Processor

Nesta parte do Webhook, **guarda o request_id associado ao ficheiro quando recebe "OK"** e **apaga o ficheiro** no **incoming/.**

```python
@app.post("/webhook/xml")
async def webhook_xml(req: Request):
    body = await req.json()
    request_id = body.get("request_id")
    status = body.get("status")
    doc_id = body.get("doc_id")

    print("WEBHOOK RECEIVED:", body)

    if request_id in pending and status == "OK":
        key = pending.pop(request_id)
        try:
            s3.delete_object(Bucket=BUCKET, Key=key)
            print(f"Deleted incoming: {key} (doc_id={doc_id})")
        except Exception as e:
            print("Delete failed:", e)

    return {"ok": True}
```

Pipeline - Processor

Prova do fluxo assíncrono: o **processor processa o CSV**, **envia para o xml-service, recebe confirmação via webhook** e **remove o ficheiro em incoming/ após persistência bem-sucedida.**

```
processor-1       | Processing: incoming/market_20260120_144543.csv
xml-service-1     | Webhook sent: http://processor:8089/webhook/xml 8d3b0b21-c6fc-45a5-b509-69b20ec0327b
processor-1       | Uploaded processed: processed/market_20260120_144543.csv
processor-1       | XML service: 200 {"status":"OK","doc_id":"a4e2c778-26aa-401a-ad70-ba0297dfe73b","request_id":"8d3b0b21-c6fc-45a5-b509-
69b20ec0327b"}
processor-1       | WEBHOOK RECEIVED: {'request_id': '8d3b0b21-c6fc-45a5-b509-69b20ec0327b', 'status': 'OK', 'doc_id': 'a4e2c778-26aa-401a
-ad70-ba0297dfe73b'}
processor-1       | Deleted incoming: incoming/market_20260120_144543.csv (doc_id=a4e2c778-26aa-401a-ad70-ba0297dfe73b)
processor-1       | INFO:     172.18.0.3:35786 - "POST /webhook/xml HTTP/1.1" 200 OK
```

Pipeline - XML Service

O **XML Service** é o serviço responsável por **receber o CSV enriquecido**, **converter os dados para XML**, **persistir o documento XML no PostgreSQL** e disponibilizar **consultas ao XML** através de endpoints que usam XPath/XMLTABLE. É também aqui que se fecha o ciclo assíncrono, enviando a confirmação por **webhook** ao processor.

O **XML Service** recebe o CSV enriquecido através do endpoint POST /ingest, faz o parsing do ficheiro e converte os registos para um **XML hierárquico**. De seguida, persiste esse documento na base de dados, respondendo em **JSON** com o estado da operação e enviando um **webhook** de confirmação para o processor quando a gravação é bem-sucedida. Além disso, disponibiliza endpoints de consulta que extraem informação do XML guardado através de **XPath/XMLTABLE**, permitindo listar ativos, obter agregações por categoria e calcular o top por volume.

Pipeline - XML Service

No código deste serviço, é importante ressaltar: **no endpoint /ingest, recebe um ficheiro e a respetiva metadata (mapper_version, request_id, webhook_url),** além de integrar com o processor.

```
// multipart: file, mapper_version, request_id, webhook_url
app.post("/ingest", upload.single("file"), async (req, res) => {
  const mapperVersion = req.body.mapper_version || "v1";
  const requestId = req.body.request_id || "no-request-id";
  const webhookUrl = req.body.webhook_url || null;

  if (!req.file) return res.status(400).json({ error: "missing file" });

  try {
    const rows = await parseCsv(req.file.buffer);
    const xml = buildXml(rows, mapperVersion);

    const q = `
      INSERT INTO tp3_documentos_xml(xml_documento, mapper_version, request_id)
      VALUES ($1::xml, $2, $3)
      RETURNING id
    `;
    const r = await pool.query(q, [xml, mapperVersion, requestId]);
    const docId = r.rows[0].id;

    // Resposta imediata ao caller
    res.json({ status: "OK", doc_id: docId, request_id: requestId });

    // Webhook assíncrono (não bloqueia o response)
    if (webhookUrl) {
      sendWebhook(webhookUrl, {
        request_id: requestId,
        status: "OK",
        doc_id: docId,
```

Pipeline - XML Service

A função **parseCSV converte o buffer do upload em registo JSON para construir o XML.**

```
function parseCsv(buffer) {
  return new Promise((resolve, reject) => {
    const records = [];
    const parser = parse(buffer, {
      columns: true,
      skip_empty_lines: true,
      trim: true,
    });

    parser.on("readable", () => {
      let record;
      while ((record = parser.read()) !== null) records.push(record);
    });

    parser.on("error", reject);
    parser.on("end", () => resolve(records));
  });
}
```

Pipeline - XML Service

A função ***buildXML*** serve para **mostrar a hierarquia do XML.**

```
function buildXml(rows, mapperVersion) {
  const root = create({ version: "1.0" })
    .ele("RelatorioConformidade", {
      DataGeracao: new Date().toISOString(),
      Versao: "1.0",
    })
    .ele("Configuracao", { MapperVersion: mapperVersion, FonteCSV: "incoming" })
    .up()
    .ele("Ativos");

  for (const r of rows) {
    // CSV: internal_id,symbol,category,price,volume,avg30d,max6m
    const ativo = root.ele("Ativo", {
      IdInterno: r.internal_id,
      Ticker: r.symbol,
      Tipo: r.category,
    });

    // DetalheNegociacao
    ativo
      .ele("DetalheNegociacao")
      .ele("PrecoAtual", { Moeda: "EUR" })
      .txt(String(r.price))
      .up()
      .ele("VolumeNegociado", { Unidade: "Acoes" })
      .txt(String(r.volume))
      .up()
      .up();
```

Pipeline - XML Service

A tabela tp3_documentos_xml armazena o documento em formato **XML** e a respetiva **metadata**.

```sql
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS tp3_documentos_xml (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  xml_documento XML NOT NULL,
  data_criacao TIMESTAMP NOT NULL DEFAULT NOW(),
  mapper_version TEXT NOT NULL,
  request_id TEXT NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_tp3_request_id ON tp3_documentos_xml(request_id);
```

Pipeline - XML Service

Evidência de persistência: **a tabela tp3_documentos_xml contém documentos XML inseridos** (contagem total) e **respetiva metadata** (id, mapper_version, request_id, data_criacao).

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> docker compose exec postgres psql -U postgres -d tp3 -c "select count(*) from tp3_documentos_xml;"
 count
-------
  1641
(1 row)

PS C:\Uni\3º ano\1º Semestre\IS\tp3> docker compose exec postgres psql -U postgres -d tp3 -c "select id, mapper_version, request_id, data_
criacao from tp3_documentos_xml order by data_criacao desc limit 5;"
                  id                  | mapper_version |              request_id              |        data_criacao
--------------------------------------+----------------+--------------------------------------+----------------------------
 f8f80c4c-c51a-4250-9bdd-7e74cc6e849d | v1             | 1c21c858-6017-455a-a3fc-50f8678fe856 | 2026-01-20 15:00:59.075289
 988dd580-9cac-41aa-a47c-a961824bfda6 | v1             | 156ac0ad-22e7-48fa-8a71-27f20e9df339 | 2026-01-20 15:00:48.567243
 b3b67ed3-f3af-4973-9c41-f0940d505041 | v1             | d3854264-4b8e-412f-9f20-8da7fa81827f | 2026-01-20 15:00:27.790994
 c0b72ce3-bf6b-42c8-817b-89c67bd7c4fc | v1             | 7edeb3ad-7519-4c97-9875-abd7d84ced24 | 2026-01-20 15:00:17.269945
 3598857a-5799-478e-a22d-5ff8bfff49a7 | v1             | d710f8ed-2c63-4f07-a0a4-daf588a150be | 2026-01-20 15:00:06.7127
(5 rows)
```

Pipeline - XML Service

Evidência do XML Service: **os endpoints REST respondem com sucesso (/health)** e **disponibilizam consultas em JSON** (/query/ativos, /query/agg/category, /query/top-volume) **extraídas do XML persistido** via XPath/XMLTABLE.

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> irm http://localhost:3000/health
>> irm "http://localhost:3000/query/ativos"
>> irm "http://localhost:3000/query/agg/category"
>> irm "http://localhost:3000/query/top-volume?limit=5"

  ok
  --
True
```

Pipeline - XML Service

O resultado do comando seguinte, **confirma que o documento XML é guardado na coluna xml_documento e contém hierarquia.**

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> docker compose exec postgres psql -U postgres -d tp3 -c "SELECT LEFT(xml_documento::text, 400) AS xml_excerpt FROM tp3_documentos_xml ORDER BY data_criacao DESC LIMIT 1;"
                                xml_excerpt
-------------------------------------------------------------------------------
 <?xml version="1.0"?>                                                        +
 <RelatorioConformidade DataGeracao="2026-01-20T20:08:23.891Z" Versao="1.0">+
   <Configuracao MapperVersion="v1" FonteCSV="incoming"/>                     +
   <Ativos>                                                                   +
     <Ativo IdInterno="CSV_3001" Ticker="GOOGL" Tipo="Tecnologia">            +
       <DetalheNegociacao>                                                    +
         <PrecoAtual Moeda="EUR">104.04</PrecoAtual>                          +
         <VolumeNegociado Unidade="Acoes">842350</VolumeNegociado>            +
       </DetalheNegociaca
(1 row)
```

Pipeline - XPath/XMLTABLE

O XPath permite **navegar** pelos **documentos XML**. No PostgreSQL, a **função XMLTABLE aplica XPath ao XML guardado e transforma nós/atributos em colunas, permitindo filtrar, agregar e ordenar.**

O **XML é persistido na tabela tp3_documentos_xml** e, através da função XMLTABLE, esse **XML é convertido em linhas e colunas consultáveis em SQL**. Esta abordagem é a **base das três consultas principais do sistema**, permitindo listar ativos, obter agregações por categoria e calcular o top por volume.

Pipeline - XPath/XMLTABLE

Query SQL com XMLTABLE: usa **XPath para extrair campos do XML persistido e convertê-los em colunas.**

```
const sql = `
  SELECT
    x.ticker,
    x.tipo,
    (x.preco)::float AS preco,
    (x.volume)::bigint AS volume
  FROM tp3_documentos_xml d,
  LATERAL XMLTABLE(
    '/RelatorioConformidade/Ativos/Ativo'
    PASSING d.xml_documento
    COLUMNS
      ticker text PATH '@Ticker',
      tipo   text PATH '@Tipo',
      preco  text PATH 'DetalheNegociacao/PrecoAtual/text()',
      volume text PATH 'DetalheNegociacao/VolumeNegociado/text()'
  ) AS x
  WHERE ($1::text IS NULL OR x.tipo = $1::text)
  ORDER BY x.ticker
  LIMIT 200
`;
```

Pipeline - XPath/XMLTABLE

Resultado do endpoint /query/ativos: **devolve JSON gerado a partir da extração do XML na BD via XPath/XMLTABLE.**

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> irm "http://localhost:3000/query/ativos"

ativos
------
{@{ticker=AAPL; tipo=Tecnologia; preco=189,2; volume=1200000}, @{ticker=AAPL; tipo=Tecnologia; preco=189,2; volume=1200000}, @{ticker=...
```

Pipeline - BI Service (GraphQL)

O **BI Service** é a camada de consumo: recebe pedidos do utilizador em **GraphQL** e, para responder, chama os endpoints REST do **xml-service** que já consultam o XML na BD. No final, o BI devolve os dados em **JSON** com o formato pretendido, servindo de interface única para consultas.

No main.py, a classe Query concentra as três consultas principais, **ativos(tipo)** (listagem com filtro opcional), **aggCategory()** (agregação por categoria) e **topVolume(limit)** (ranking por volume). A parte mais importante é o **helper _get_json(...),** que faz os **pedidos ao xml-service**, e os **resolvers GraphQL**, que **transformam as respostas em tipos GraphQL**, ou seja, o núcleo do BI é traduzir GraphQL → REST → JSON.

Pipeline - BI Service (GraphQL)

O **_get_json(...)** é a **função auxiliar que faz a ponte entre o BI Service e o xml-service**: **constrói o URL com base no XML_SERVICE_BASE**, **executa um GET** com requests, **valida a resposta** (raise_for_status) e **devolve o conteúdo em JSON**.

Na prática, é esta função que permite ao BI consumir os dados já extraídos do XML (via endpoints REST) sem aceder diretamente ao PostgreSQL.

```python
def _get_json(path: str, params: dict | None = None) -> dict:
    url = f"{XML_SERVICE_BASE}{path}"
    r = requests.get(url, params=params, timeout=20)
    r.raise_for_status()
    return r.json()
```

Pipeline - BI Service (GraphQL)

O ***topVolume(limit)*** é a query GraphQL que devolve os **N ativos com maior volume negociado**. Internamente, chama o endpoint /query/top-volume do xml-service com o parâmetro limit, recebe a lista ordenada por volume e converte cada item para o tipo GraphQL Ativo. Como esse endpoint não devolve o preço, o BI preenche preco=0.0 apenas para manter o mesmo tipo de retorno e um schema consistente. No ranking por volume, o preço não vem do endpoint, por isso é colocado 0.0 apenas para manter o schema GraphQL consistente.

```python
@strawberry.field
def topVolume(self, limit: int = 5) -> list[Ativo]:
    data = _get_json("/query/top-volume", params={"limit": limit})
    rows = data.get("top", [])
    out = []
    for x in rows:
        out.append(
            Ativo(
                ticker=str(x.get("ticker", "")),
                tipo=str(x.get("tipo", "")),
                preco=0.0,  # este endpoint não devolve preco; mantemos 0
                volume=int(x.get("volume", 0)),
            )
        )
    return out
```

Pipeline - BI Service (GraphQL)

De seguida segue-se uma querie GraphQL executada no BI Service que devolvem resultados em JSON.

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> $body = @{ query = '{ aggCategory { tipo total } }' } | ConvertTo-Json
>> irm -Method Post -Uri http://localhost:8000/graphql -ContentType 'application/json' -Body $body

data
----
@{aggCategory=System.Object[]}
```

Pipeline - BI Service (GraphQL)

Na imagem seguinte observa-se uma **visualização simples que consome o endpoint /graphql do BI Service**.



Pipeline - BI Service (GraphQL)

Na imagem seguinte observa-se uma **confirmação da ligação ao xml-service.**

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> docker compose logs --tail=50 bi-service
bi-service-1  | INFO:     Started server process [1]
bi-service-1  | INFO:     Waiting for application startup.
bi-service-1  | INFO:     Application startup complete.
bi-service-1  | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
bi-service-1  | INFO:     172.18.0.1:43898 - "POST /graphql HTTP/1.1" 200 OK
bi-service-1  | INFO:     172.18.0.1:57222 - "POST /graphql HTTP/1.1" 200 OK
bi-service-1  | INFO:     172.18.0.1:53836 - "GET / HTTP/1.1" 200 OK
bi-service-1  | INFO:     172.18.0.1:53836 - "GET /favicon.ico HTTP/1.1" 404 Not Found
bi-service-1  | INFO:     172.18.0.1:53836 - "POST /graphql HTTP/1.1" 200 OK
```

Pipeline – Viewer Service

O **Viewer Service** é um serviço em **Go** que funciona como camada de visualização/consumo do sistema.

No trabalho, é usado para fornecer uma forma simples e direta de consultar os dados sem recorrer ao terminal. Em termos de funcionamento, o **viewer** faz pedidos ao **BI Service** através de **GraphQL**, obtendo os mesmos resultados das queries principais (listagem de ativos, agregação por categoria e top por volume) e apresentando-os numa interface web. Assim, o **viewer não acede à base de dados nem ao XML diretamente**: consome a API GraphQL do BI, que por sua vez consulta o xml-service.

Pipeline – Viewer Service

Embora tenha a mesma interface que o Bi Service, a **página do Viewer Service foi criada através da linguagem de programação Go**, de forma a ser fácil e intuitivo.



Pipeline – Interfaces de Interoperabilidade

Depois de criar o núcleo do sistema (XML persistido + consultas REST/GraphQL), **adiciona-se interfaces adicionais para consumo por diferentes tipos de clientes**. O **XML-RPC garante compatibilidade e simplicidade**, enquanto o **gRPC fornece um contrato forte** e **chamadas remotas** eficientes.

O XML-RPC é um protocolo RPC mais simples e tradicional. No trabalho, **o xmlrpc-service expõe métodos remotos** (ping, agg_category, top_volume) e, internamente, **chama os endpoints REST do xml-service**, **devolvendo os resultados ao cliente no formato XML-RPC.**

Pipeline – Interfaces de Interoperabilidade

No código do xml-rpc, pode notar-se a **utilização de métodos mais antigos.**

```python
def ping():
    return "pong"

def agg_category():
    r = requests.get(f"{XML_SERVICE_BASE}/query/agg/category", timeout=15)
    r.raise_for_status()
    return r.json()  # retorna dict (XML-RPC suporta structs)

def top_volume(limit=5):
    r = requests.get(f"{XML_SERVICE_BASE}/query/top-volume", params={"limit": int(limit)}, timeout=15)
    r.raise_for_status()
    return r.json()
```

Pipeline – Interfaces de Interoperabilidade

Por fim, são notórias as **chamadas do XML-RPC** (ping, agg_category, top_volume) a **devolver resultados com sucesso**.

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> python -c "import xmlrpc.client as x; s=x.ServerProxy('http://localhost:9001'); print('ping:', s.ping
()); print('agg:', s.agg_category()); print('top:', s.top_volume(5))"
ping: pong
agg: {'agg': [{'tipo': 'Tecnologia', 'total': 4308}, {'tipo': 'Automovel', 'total': 2154}]}
top: {'top': [{'ticker': 'AAPL', 'tipo': 'Tecnologia', 'volume': '1200000'}, {'ticker': 'AAPL', 'tipo': 'Tecnologia', 'volume': '1200000'}
, {'ticker': 'AAPL', 'tipo': 'Tecnologia', 'volume': '1200000'}, {'ticker': 'AAPL', 'tipo': 'Tecnologia', 'volume': '1200000'}, {'ticker':
 'AAPL', 'tipo': 'Tecnologia', 'volume': '1200000'}]}
```

Pipeline – Interfaces de Interoperabilidade

Já o gRPC, é um **protocolo RPC moderno baseado em Protobuf** e **contratos definidos em ficheiros .proto**. No trabalho, o **grpc-service define o contrato (ativos.proto) e disponibiliza métodos como GetAtivos, GetAggCategory e GetTopVolume**, consumindo internamente, o **xml-service para obter os dados** e **devolvendo respostas tipadas** ao cliente.

Pipeline – Interfaces de Interoperabilidade

Neste pequeno excerto do **ficheiro .proto**, pode-se observar um **tipo de mensagem e um serviço.**

```
message TopResponse {
  repeated Ativo top = 1;
}

service Tp3Service {
  rpc Ping(Empty) returns (Empty);
  rpc GetAtivos(Empty) returns (AtivosResponse);
  rpc GetAggCategory(Empty) returns (AggResponse);
  rpc GetTopVolume(TopRequest) returns (TopResponse);
}
```

Pipeline – Interfaces de Interoperabilidade

É dentro da classe ***Tp3Service***, que serão implementados os RPC's. A chamada do get_json prova que é um adaptador para o xml-service.

```python
class Tp3Service(ativos_pb2_grpc.Tp3ServiceServicer):
    def Ping(self, request, context):
        return ativos_pb2.Empty()

    def GetAtivos(self, request, context):
        data = get_json("/query/ativos")
        rows = data.get("ativos", [])
        resp = ativos_pb2.AtivosResponse()
        for x in rows:
            resp.ativos.append(
                ativos_pb2.Ativo(
                    ticker=str(x.get("ticker", "")),
                    tipo=str(x.get("tipo", "")),
                    preco=float(x.get("preco", 0) or 0),
                    volume=int(x.get("volume", 0) or 0),
                )
            )
        return resp
```

Pipeline – Interfaces de Interoperabilidade

É dentro desta função, que **será criado o server** e será **executado o add_insecure_port(...).**

```
def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    ativos_pb2_grpc.add_Tp3ServiceServicer_to_server(Tp3Service(), server)
    server.add_insecure_port(f"0.0.0.0:{GRPC_PORT}")
    server.start()
    print(f"grpc-service on :{GRPC_PORT}")
    server.wait_for_termination()
```

Pipeline – Interfaces de Interoperabilidade

Na imagem seguinte é possível ver **as chamadas gRPC** aos **métodos definidos em ativos.proto** a **devolver respostas tipadas** (Agg e TopVolume).

```
PS C:\Uni\3º ano\1º Semestre\IS\tp3> docker compose exec grpc-service python -c "import ativos_pb2 as pb, ativos_pb2_grpc as g; import grpc; ch=grpc.insecure_channel('localhost:50051'); s=g.Tp3ServiceStub(ch); print('Ping:', s.Ping(pb.Empty())); print('Agg:', s.GetAggCategory(pb.Empty())); print('Top:', s.GetTopVolume(pb.TopRequest(limit=5)))"
Ping:
Agg: agg {
  tipo: "Tecnologia"
  total: 4556
}
agg {
  tipo: "Automovel"
  total: 2278
}

Top: top {
  ticker: "AAPL"
  tipo: "Tecnologia"
  volume: 1200000
}
```

Pipeline – Docker

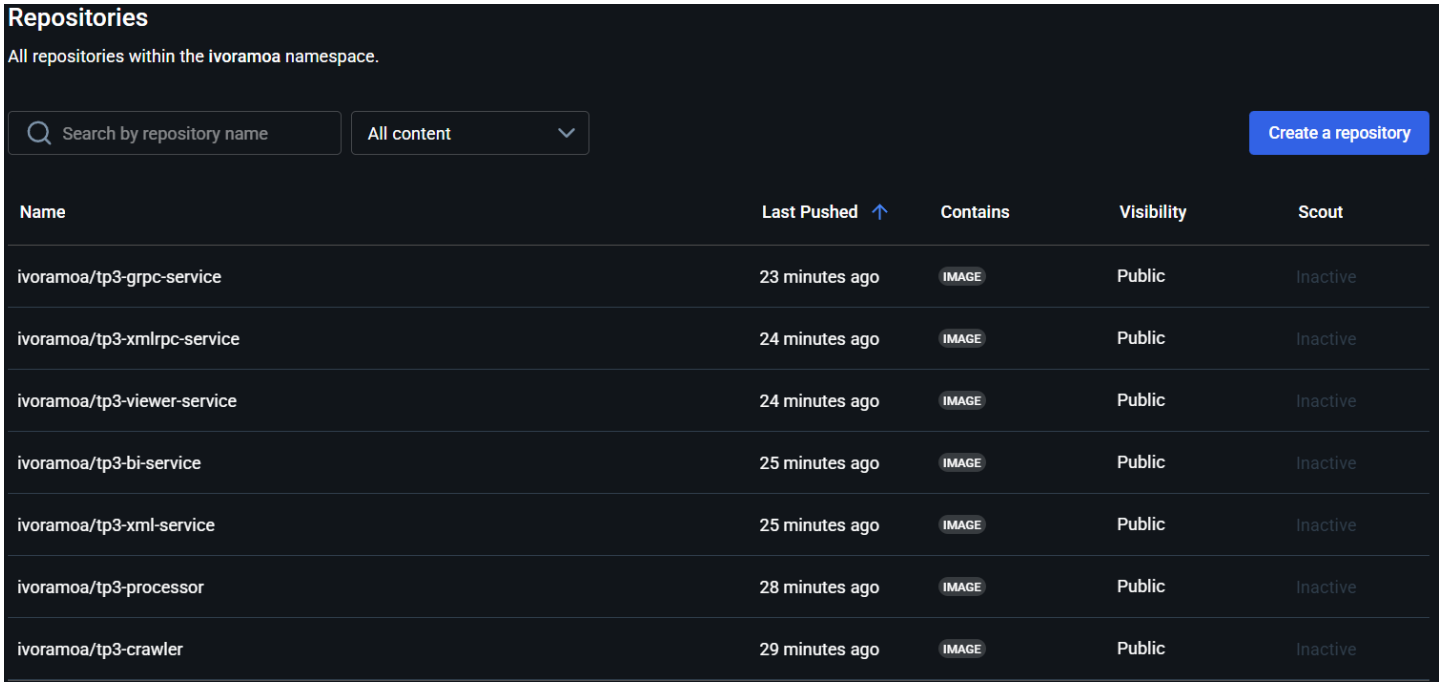
O projeto é executado via **Docker Compose**, com 8 containers a correr em simultâneo: **cada microserviço em container próprio** mais a **base de dados Postgres.** Isto garante isolamento, reprodutibilidade e execução consistente em qualquer máquina.



tp3	-	-	-	1.58%	46 minutes ago	
postgres-1	a1082111164a	postgres:16	5432:5432	0.03%	2 hours ago	
xml-service-1	f782194f3b55	tp3-xml-service	3000:3000	0%	46 minutes ago	
processor-1	0f56e65d32b0	tp3-processor	8089:8089	1.07%	46 minutes ago	
xmlrpc-service-1	83a50ed96bf6	tp3-xmlrpc-service	9001:9001	0.02%	46 minutes ago	
bi-service-1	c658671ff63e	tp3-bi-service	8000:8000	0.25%	46 minutes ago	
grpc-service-1	52b68552ad53	tp3-grpc-service	50051:50051	0.21%	46 minutes ago	
crawler-1	274ba3d3eb49	tp3-crawler		0%	46 minutes ago	
viewer-service-1	0e5f2c943c62	tp3-viewer-service	8080:8080	0%	46 minutes ago	

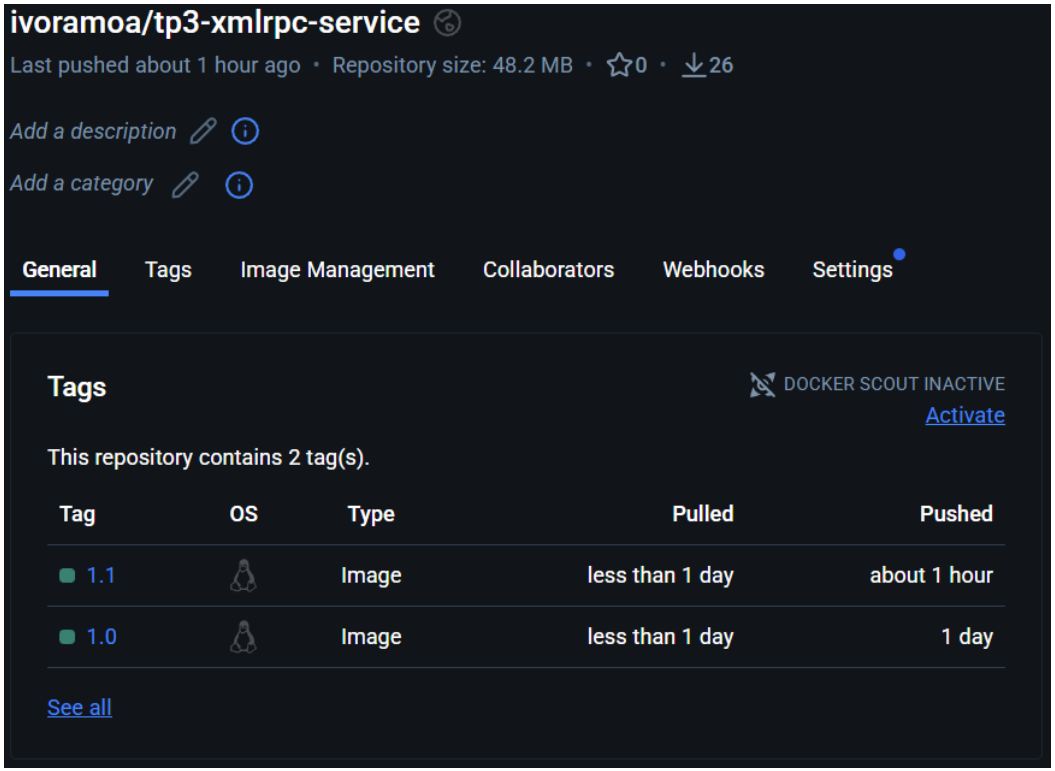
Pipeline – Docker

Já no DockerHub, as **imagens Docker dos serviços foram versionadas** e **publicadas no Docker Hub (tag 1.1)**, permitindo distribuir e executar a solução noutros ambientes sem necessidade de compilar localmente.



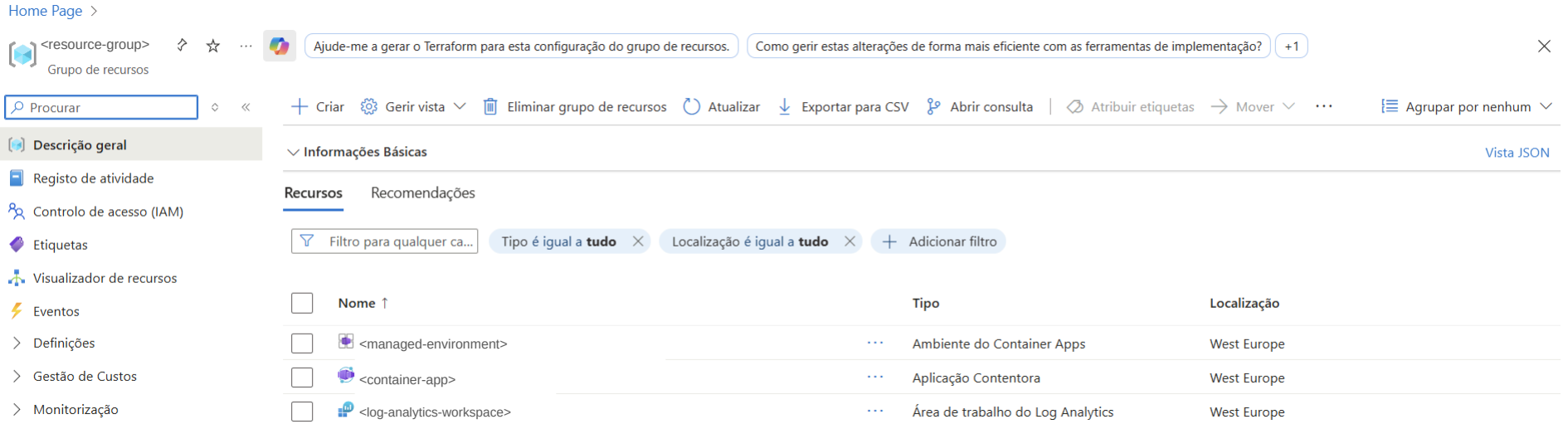
Pipeline – Docker

Ao entrar numa das imagens no DockerHub, poderá **ver-se detalhes sobre a imagem**, **como o número da versão.**



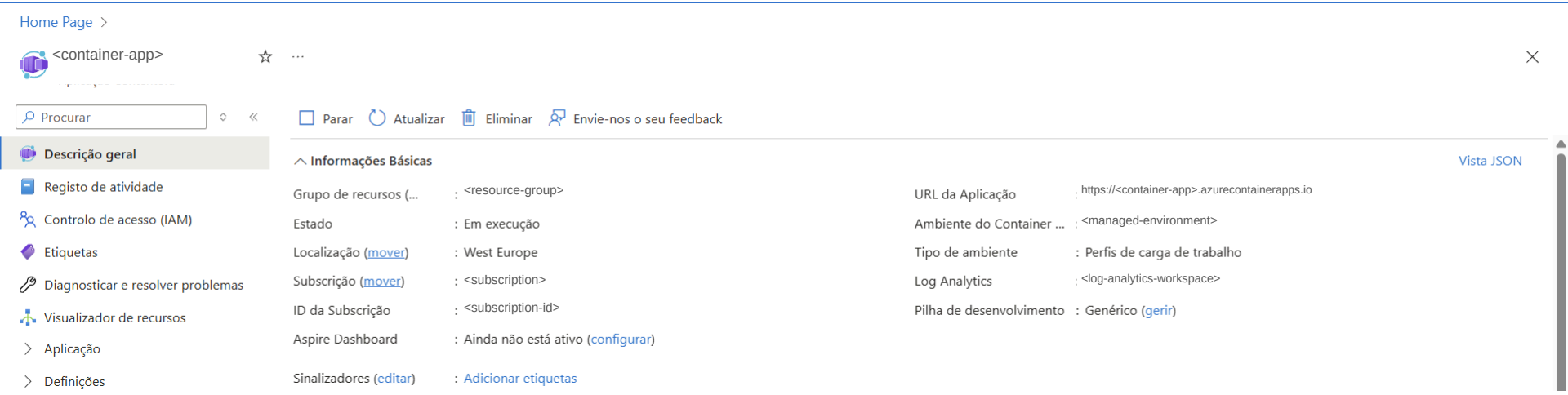
Pipeline - Azure

Na componente Cloud, foi usado o Microsoft Azure para disponibilizar publicamente o **xml-service**. Primeiro foi criado um **Resource Group (RG)** para agrupar todos os recursos do projeto e facilitar a gestão.



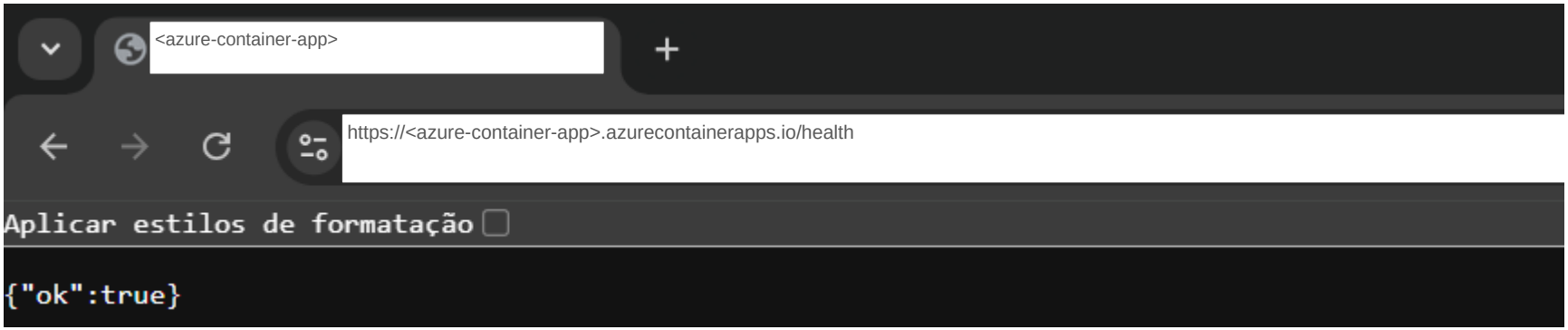
Pipeline - Azure

De seguida, foi criada uma **Container App** e **configurada para executar o serviço xml-service.**



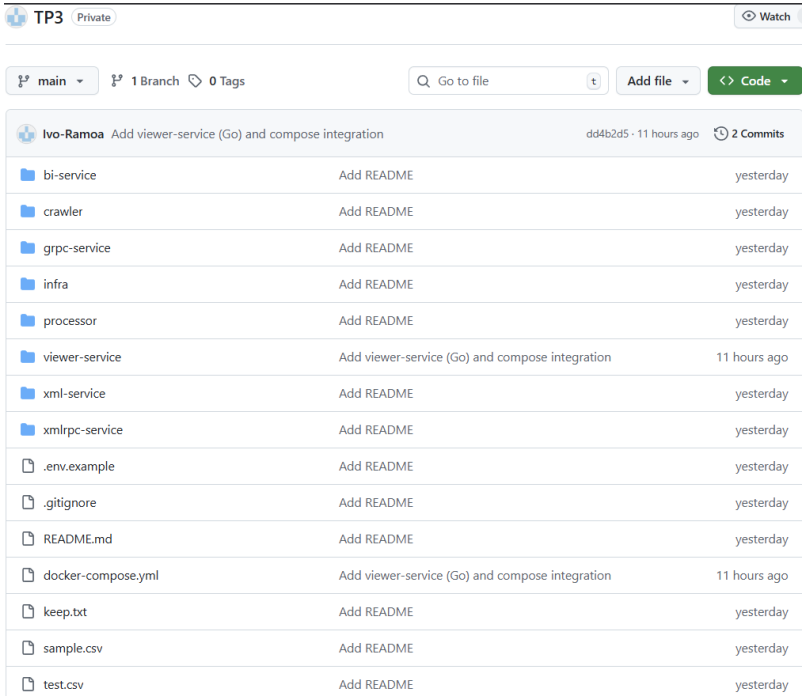
Pipeline - Azure

O **serviço ficou exposto por HTTP** e foi **validado através do endpoint /health**, **confirmando** que **o container está ativo e acessível na cloud.**



Pipeline - GitHub

Todo o código-fonte do TP3 foi versionado num **repositório GitHub,** incluindo a **estrutura por serviços, Dockerfiles e o docker-compose.yml**. Foram feitos commits incrementais ao longo do desenvolvimento para registar alterações e evolução do sistema, garantindo rastreabilidade.



Fonte: https://github.com/Ivo-Ramoa/TP3.git

Conclusão

Este trabalho implementou uma solução **de Integração de Sistemas baseada em microserviços**, com um **pipeline completo desde a geração de CSV e armazenamento em Supabase (S3)**, **passando pelo processamento e enriquecimento com uma API externa, até à conversão para XML e persistência em PostgreSQL**. Os dados foram depois disponibilizados por **várias interfaces e protocolos (REST, GraphQL, XML-RPC e gRPC), garantindo interoperabilidade e flexibilidade de consumo.** A execução **containerizada com Docker Compose, bem como o versionamento no GitHub e Docker Hub, asseguraram reprodutibilidade** e rastreabilidade da entrega.

Bibliografia e Referências Web

- Repositório GitHub:https://github.com/Ivo-Ramoa/TP3.git
- Microsoft. (n.d.). Deploy an existing container image to Azure Container Apps. Microsoft Learn.https://learn.microsoft.com/en-us/azure/container-apps/get-started-existing-container-image-portal?pivots=container-apps-private-registry
- Xml-rpc: Python Software Foundation. (n.d.). xmlrpc.client — XML-RPC client access. Python Documentation.https://docs.python.org/3/library/xmlrpc.client.html
- gRPC: gRPC. (n.d.). gRPC Python documentation. gRPC.io. https://grpc.io/docs/languages/python/

o teu • de partida



www.ipvc.pt